

MCY 612 Cloud Security Assignment 2

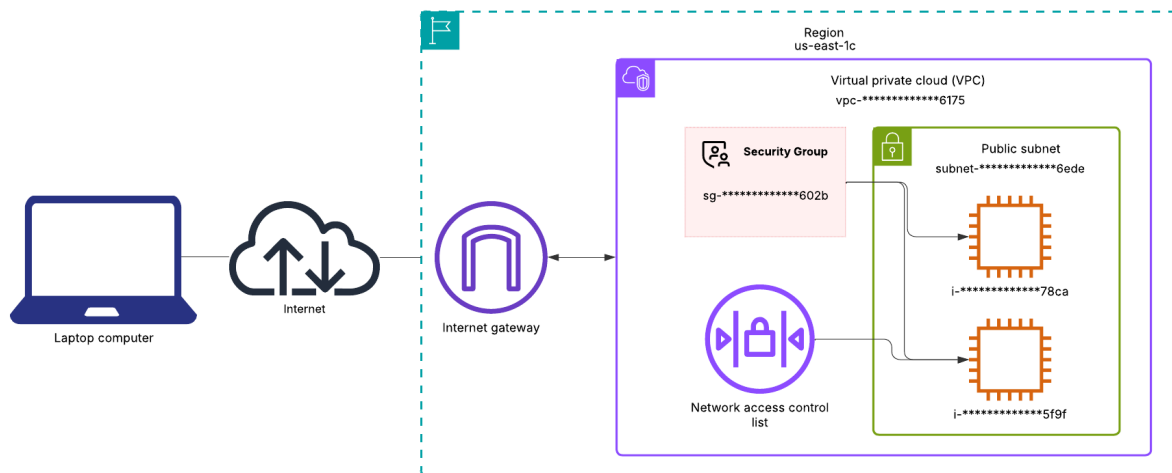
DQuez Carr
Cloud Security
Wei Hao
March, 2025

Report

Introduction

1. You need to clearly describe an example, which shows one difference between the security group and the NACL. Network ACLs and Security Groups let you manage who can access AWS resources inside your VPC. However, network ACLs provide similar advantages at the VPC subnet level, while security groups let you manage incoming and outgoing traffic at the instance level.

2. You need to draw an AWS architecture diagram to illustrate how you implement the example. You need to build your architecture diagrams with AWS-approved icons at <https://aws.amazon.com/architecture/icons/>.



Materials

AWS Cloudshell terminal

AWS CLI

AWS Version

aws-cli/2.23.13 Python/3.12.6 Linux/6.1.127-135.201.amzn2023.x86_64 exec-env/CloudShell

exe/x86_64.amzn.2023

AWS Configure

AWS Access Key ID [*****FLZO2]

AWS Secret Access Key [*****23fZK]

Default region name [us-east-1]

Default output format [json]

Security Group sg-*****602b allows incoming traffic over port 22 for SSH

VPC vpc-*****6175

Subnet ID subnet-*****6ede

Ubuntu Server 20.04 LTS AMI: ami-09e67e426f25ce0d7

Network acl acl-*****ee08

Methods

Part 1 Create a VPC and a subnet

1. Create a VPC via the following command:

```
~ $ aws ec2 create-vpc --cidr-block 10.6.8.0/26
{
  "Vpc": {
    "OwnerId": "881490118823",
    "InstanceTenancy": "default",
    "Ipv6CidrBlockAssociationSet": [],
    "CidrBlockAssociationSet": [
      {
        "AssociationId": "vpc-cidr-assoc-0075ec512b32cb75d",
        "CidrBlock": "10.6.8.0/26",
        "CidrBlockState": {
          "State": "associated"
        }
      }
    ],
    "IsDefault": false,
    "VpcId": "vpc-0cb3e30db617f6175",
    "State": "pending",
    "CidrBlock": "10.6.8.0/26",
    "DhcpOptionsId": "dopt-08b45b45933d7afb0"
  }
}
```

2. Create a subnet in the VPC

```
~ $ aws ec2 create-subnet --vpc-id vpc-0cb3e30db617f6175 --cidr-block 10.6.8.0/28 --availability-zone us-east-1c
{
  "Subnet": {
    "AvailabilityZoneId": "use1-az2",
    "OwnerId": "881490118823",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "SubnetArn": "arn:aws:ec2:us-east-1:881490118823:subnet/subnet-09df9a6645aea6ede",
    "EnableDns64": false,
    "Ipv6Native": false,
    "PrivateDnsNameOptionsOnLaunch": {
      "HostnameType": "ip-name",
      "EnableResourceNameDnsARecord": false,
      "EnableResourceNameDnsAAAARecord": false
    },
    "SubnetId": "subnet-09df9a6645aea6ede",
    "State": "available",
    "VpcId": "vpc-0cb3e30db617f6175",
    "CidrBlock": "10.6.8.0/28",
    "AvailableIpAddressCount": 11,
    "AvailabilityZone": "us-east-1c",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false
  }
}
```

3. Create an internet gateway for your VPC

```
~ $ aws ec2 create-internet-gateway
{
  "InternetGateway": {
    "Attachments": [],
    "InternetGatewayId": "igw-0cc59d6a34d65d113",
    "OwnerId": "881490118823",
    "Tags": []
  }
}
```

4. Attach the internet gateway to your VPC to enable connectivity between the internet and your VPC

```
~ $ aws ec2 attach-internet-gateway --vpc-id vpc-0cb3e30db617f6175 --internet-gateway-id igw-0cc59d6a34d65d113
```

5. Create a route table for your VPC

```

~ $ aws ec2 create-route-table --vpc-id vpc-0cb3e30db617f6175
{
  "RouteTable": {
    "Associations": [],
    "PropagatingVgws": [],
    "RouteTableId": "rtb-0940a3306905b7239",
    "Routes": [
      {
        "DestinationCidrBlock": "10.6.8.0/26",
        "GatewayId": "local",
        "Origin": "CreateRouteTable",
        "State": "active"
      }
    ],
    "Tags": [],
    "VpcId": "vpc-0cb3e30db617f6175",
    "OwnerId": "881490118823"
  },
  "ClientToken": "b49e4e04-1b7f-4d8d-9870-08f4028835b7"
}
~ $

```

6. Create a networking route in your route table within your VPC

```

~ $ aws ec2 create-route --route-table-id rtb-0940a3306905b7239 --destination-cidr-block 0.0.0.0/0 --gateway-id igw-0cc59d6a34d65d113
{
  "Return": true
}
~ $

```

7. Verify your route has been created and is active

```

~ $ aws ec2 describe-route-tables --route-table-id rtb-0940a3306905b7239
{
  "RouteTables": [
    {
      "Associations": [],
      "PropagatingVgws": [],
      "RouteTableId": "rtb-0940a3306905b7239",
      "Routes": [
        {
          "DestinationCidrBlock": "10.6.8.0/26",
          "GatewayId": "local",
          "Origin": "CreateRouteTable",
          "State": "active"
        },
        {
          "DestinationCidrBlock": "0.0.0.0/0",
          "GatewayId": "igw-0cc59d6a34d65d113",
          "Origin": "CreateRoute",
          "State": "active"
        }
      ],
      "Tags": [],
      "VpcId": "vpc-0cb3e30db617f6175",
      "OwnerId": "881490118823"
    }
  ]
}
~ $

```

8. Associate the subnet with the route table in your VPC. This association causes traffic from your subnet to be routed according to the routes in your route table.

```

~ $ aws ec2 associate-route-table --subnet-id subnet-09df9a6645aea6ede --route-table-id rtb-0940a3306905b7239
{
  "AssociationId": "rtbassoc-0ca3a6f2140910dbe",
  "AssociationState": {
    "State": "associated"
  }
}
~ $

```

9. Modify the public IP addressing attribute with the following command so that the EC2 Instance launched into your subnet automatically receives a public IP address.

```

~ $ aws ec2 modify-subnet-attribute --subnet-id subnet-09df9a6645aea6ede --map-public-ip-on-launch
~ $

```

10. Add a rule, which allows incoming traffic over port 22 for SSH, to the security group

```
~ $ aws ec2 authorize-security-group-ingress --group-id sg-0b1cfa43fd7ea602b --protocol tcp --port 22 --cidr 0.0.0.0/0
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-034a3b837fe227642",
      "GroupId": "sg-0b1cfa43fd7ea602b",
      "GroupOwnerId": "881490118823",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "0.0.0.0/0",
      "SecurityGroupRuleArn": "arn:aws:ec2:us-east-1:881490118823:security-group-rule/sgr-034a3b837fe227642"
    }
  ]
}
```

11. Create a key pair

```
~ $ aws ec2 create-key-pair --key-name carrd12-key --query 'KeyMaterial' --output text > devenv-key.pem
```

Modify permissions to the devenv-key.pem file with the following command so that only i can access it

```
chmod 400 devenv-key.pem
```

12. Launch an EC2 instance into your VPC

```
~ $ aws ec2 run-instances --image-id ami-09e67e426f25ce0d7 --count 1 --instance-type t3.micro --key-name carrd12-key --security-group-ids sg-0b1cfa43fd7ea602b --subnet-id subnet-09df9a6645aea6ede
{
  "ReservationId": "r-097dbfde1cf51977e",
  "OwnerId": "881490118823",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "e9dec8e2-e9ea-4365-a609-d518e3a54634",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2025-03-29T01:31:40+00:00",
            "AttachmentId": "eni-attach-0ea6d284323a0df35",
            "DeleteOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",

```

```

        "Status": "attaching",
        "NetworkCardIndex": 0
    },
    "Description": "",
    "Groups": [
        {
            "GroupId": "sg-0b1cfa43fd7ea602b",
            "GroupName": "carrd12"
        }
    ],
    "Ipv6Addresses": [],
    "MacAddress": "12:a6:ad:3b:d1:bf",
    "NetworkInterfaceId": "eni-030f0b1eac10daf70",
    "OwnerId": "881490118823",
    "PrivateIpAddress": "10.6.8.13",
    "PrivateIpAddresses": [
        {
            "Primary": true,
            "PrivateIpAddress": "10.6.8.13"
        }
    ],
    "SourceDestCheck": true,
    "StateReason": {

```

```

        "SourceDestCheck": true,
        "Status": "in-use",
        "SubnetId": "subnet-09df9a6645aea6ede",
        "VpcId": "vpc-0cb3e30db617f6175",
        "InterfaceType": "interface",
        "Operator": {
            "Managed": false
        }
    },
    "RootDeviceName": "/dev/sda1",
    "RootDeviceType": "ebs",
    "SecurityGroups": [
        {
            "GroupId": "sg-0b1cfa43fd7ea602b",
            "GroupName": "carrd12"
        }
    ],
    "SourceDestCheck": true,
    "StateReason": {
        "Code": "pending",
        "Message": "pending"
    }
}

```



```

    "PublicDnsName": "",
    "StateTransitionReason": "",
    "KeyName": "carrd12-key",
    "AmiLaunchIndex": 0,
    "ProductCodes": [],
    "InstanceType": "t3.micro",
    "LaunchTime": "2025-03-29T01:31:40+00:00",
    "Placement": {
      "GroupName": "",
      "Tenancy": "default",
      "AvailabilityZone": "us-east-1c"
    },
    "Monitoring": {
      "State": "disabled"
    },
    "SubnetId": "subnet-09df9a6645aea6ede",
    "VpcId": "vpc-0cb3e30db617f6175",
    "PrivateIpAddress": "10.6.8.13"
  }
]
}
~ $ █

```

Part 2: NACL (Network Access Control List)

13. Find out the default NACL for your subnet

```

~ $ aws ec2 describe-network-acls --region us-east-1 --output table --query 'NetworkAcls[*].NetworkAclId' --filters Name=association.subnet-id,Values=subnet-09df9a6645aea6ede
+-----+-----+
| DescribeNetworkAcls |
+-----+-----+
| acl-091a9c3657de4ee08 |
+-----+-----+
~ $ █

```

14. Describe my network ACL

```

~ $ aws ec2 describe-network-acls --network-acl-ids acl-091a9c3657de4ee08
{
  "NetworkAcls": [
    {
      "Associations": [
        {
          "NetworkAclAssociationId": "aclassoc-06cc48ec7eb62debd",
          "NetworkAclId": "acl-091a9c3657de4ee08",
          "SubnetId": "subnet-09df9a6645aea6ede"
        }
      ],
      "Entries": [
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "allow",
          "RuleNumber": 100
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "deny",
          "RuleNumber": 32767
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": false,
          "Protocol": "-1",
          "RuleAction": "allow",
          "RuleNumber": 100
        }
      ],
    }
  ]
}

```

```

    },
    {
      "CidrBlock": "0.0.0.0/0",
      "Egress": false,
      "Protocol": "-1",
      "RuleAction": "deny",
      "RuleNumber": 32767
    }
  ],
  "IsDefault": true,
  "NetworkAclId": "acl-091a9c3657de4ee08",
  "Tags": [],
  "VpcId": "vpc-0cb3e30db617f6175",
  "OwnerId": "881490118823"
}
]

```

15. Create an instance in my subnet

```
-- $ aws ec2 run-instances --image-id ami-05c9c0fab0c0bb43e --count 1 --instance-type t3.micro --key-name carrd12-key --security-group-ids sg-0b1cfa43fd7ea602b --subnet-id subnet-09df9a6645aea6ede
{
  "ReservationId": "r-070c9f933b52e1e0c",
  "OwnerId": "881490118823",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "ebe6e89b-69fc-47f8-9c25-efdb72aac558",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2025-03-29T01:49:22+00:00",
            "AttachmentId": "eni-attach-02696a0e487f0e271",
            "DeleteOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",
            "NetworkCardIndex": 0
          },
          "Description": "",
          "Groups": [
            {
              "GroupId": "sg-0b1cfa43fd7ea602b",
              "GroupName": "carrd12"
            }
          ],
          "Ipv6Addresses": [],
          "MacAddress": "12:28:60:62:1b:fb",
          "NetworkInterfaceId": "eni-0dd878b14ee194ca3",

```

```
          "NetworkInterfaceId": "eni-0dd878b14ee194ca3",
          "OwnerId": "881490118823",
          "PrivateIpAddress": "10.6.8.5",
          "PrivateIpAddresses": [
            {
              "Primary": true,
              "PrivateIpAddress": "10.6.8.5"
            }
          ],
          "SourceDestCheck": true,
          "Status": "in-use",
          "SubnetId": "subnet-09df9a6645aea6ede",
          "VpcId": "vpc-0cb3e30db617f6175",
          "InterfaceType": "interface",
          "Operator": {
            "Managed": false
          }
        }
      ],
      "RootDeviceName": "/dev/sda1",
      "RootDeviceType": "ebs",
      "SecurityGroups": [
        {
          "GroupId": "sg-0b1cfa43fd7ea602b",
          "GroupName": "carrd12"
        }
      ],
      "SourceDestCheck": true,
      "StateReason": {
        "Code": "pending",
        "Message": "pending"
      },
      "VirtualizationType": "hvm",
      "CpuOptions": {

```

```

    "CpuOptions": {
      "CoreCount": 1,
      "ThreadsPerCore": 2
    },
    "CapacityReservationSpecification": {
      "CapacityReservationPreference": "open"
    },
    "MetadataOptions": {
      "State": "pending",
      "HttpTokens": "optional",
      "HttpPutResponseHopLimit": 1,
      "HttpEndpoint": "enabled",
      "HttpProtocolIpv6": "disabled",
      "InstanceMetadataTags": "disabled"
    },
    "EnclaveOptions": {
      "Enabled": false
    },
    "PrivateDnsNameOptions": {
      "HostnameType": "ip-name",
      "EnableResourceNameDnsARecord": false,
      "EnableResourceNameDnsAAAARecord": false
    },
    "MaintenanceOptions": {
      "AutoRecovery": "default"
    },
    "CurrentInstanceBootMode": "legacy-bios",
    "Operator": {
      "Managed": false
    },
    "InstanceId": "i-07376b78aa6f65f9f",
    "ImageId": "ami-05c9c0fab0c0bb43e",
    "State": {
      "Code": 0,

```

```

    "ImageId": "ami-05c9c0fab0c0bb43e",
    "State": {
      "Code": 0,
      "Name": "pending"
    },
    "PrivateDnsName": "ip-10-6-8-5.ec2.internal",
    "PublicDnsName": "",
    "StateTransitionReason": "",
    "KeyName": "carrd12-key",
    "AmiLaunchIndex": 0,
    "ProductCodes": [],
    "InstanceType": "t3.micro",
    "LaunchTime": "2025-03-29T01:49:22+00:00",
    "Placement": {
      "GroupName": "",
      "Tenancy": "default",
      "AvailabilityZone": "us-east-1c"
    },
    "Monitoring": {
      "State": "disabled"
    },
    "SubnetId": "subnet-09df9a6645aea6ede",
    "VpcId": "vpc-0cb3e30db617f6175",
    "PrivateIpAddress": "10.6.8.5"

```

```

  }
}
~ $ █

```

16. Create an entry (a rule) in your network ACL

```
~ $ aws ec2 create-network-acl-entry --network-acl-id acl-091a9c3657de4ee08 --ingress --rule-number 80 --protocol tcp --port-range From=22,To=22 --cidr-block 0.0.0.0/0 --rule-action deny
```

17. Verify if the new rule is added to the default NACL

```
~ $ aws ec2 describe-network-acls --network-acl-ids acl-091a9c3657de4ee08
{
  "NetworkAcls": [
    {
      "Associations": [
        {
          "NetworkAclAssociationId": "aclassoc-06cc48ec7eb62debd",
          "NetworkAclId": "acl-091a9c3657de4ee08",
          "SubnetId": "subnet-09df9a6645aea6ede"
        }
      ],
      "Entries": [
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "allow",
          "RuleNumber": 100
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "deny",
          "RuleNumber": 32767
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": false,
          "PortRange": {
            "From": 22,
            "To": 22
          },
          "Protocol": "6",

```

```

        "CidrBlock": "0.0.0.0/0",
        "Egress": false,
        "PortRange": {
            "From": 22,
            "To": 22
        },
        "Protocol": "6",
        "RuleAction": "deny",
        "RuleNumber": 80
    },
    {
        "CidrBlock": "0.0.0.0/0",
        "Egress": false,
        "Protocol": "-1",
        "RuleAction": "allow",
        "RuleNumber": 100
    },
    {
        "CidrBlock": "0.0.0.0/0",
        "Egress": false,
        "Protocol": "-1",
        "RuleAction": "deny",
        "RuleNumber": 32767
    }
],
    "IsDefault": true,
    "NetworkAclId": "acl-091a9c3657de4ee08",
    "Tags": [],
    "VpcId": "vpc-0cb3e30db617f6175",
    "OwnerId": "881490118823"
}
]
}
~ $

```

Testing

Do the following steps to test if the VPC, subnet, routing table you created at the previous steps

Create a security group

```

~ $ aws ec2 create-security-group --group-name carrd12 --description "my security group" --vpc-id vpc-0cb3e30db617f6175
{
    "GroupId": "sg-0b1cfa43fd7ea602b",
    "SecurityGroupArn": "arn:aws:ec2:us-east-1:881490118823:security-group/sg-0b1cfa43fd7ea602b"
}
~ $

```

Use the SSH command to log into your instance and then run "ping -c 5 google.com"

The purpose of the ping command is to make sure the instance in your VPC has the internet connection

```
~ $ ssh -i devenv-key.pem ubuntu@52.87.163.233
The authenticity of host '52.87.163.233 (52.87.163.233)' can't be established.
ED25519 key fingerprint is SHA256:LYKZxvclQ9XY7cUupzTmP33L7nP06BuVPazvjNtB66I.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '52.87.163.233' (ED25519) to the list of known hosts.
Welcome to Ubuntu 20.04.2 LTS (GNU/Linux 5.4.0-1045-aws x86_64)
```

```
* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:      https://ubuntu.com/advantage
```

System information as of Sat Mar 29 01:38:59 UTC 2025

```
System load: 0.0          Processes:            104
Usage of /:  16.4% of 7.69GB Users logged in:       0
Memory usage: 23%        IPv4 address for ens5: 10.6.8.13
Swap usage:  0%
```

1 update can be applied immediately.
To see these additional updates run: `apt list --upgradable`

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

```
ubuntu@ip-10-6-8-13:~$
```

```
ubuntu@ip-10-6-8-13:~$ ping -c 5 google.com
PING google.com (142.251.163.100) 56(84) bytes of data:
64 bytes from wv-in-f100.1e100.net (142.251.163.100): icmp_seq=1 ttl=105 time=1.67 ms
64 bytes from wv-in-f100.1e100.net (142.251.163.100): icmp_seq=2 ttl=105 time=1.66 ms
64 bytes from wv-in-f100.1e100.net (142.251.163.100): icmp_seq=3 ttl=105 time=1.72 ms
64 bytes from wv-in-f100.1e100.net (142.251.163.100): icmp_seq=4 ttl=105 time=1.72 ms
64 bytes from wv-in-f100.1e100.net (142.251.163.100): icmp_seq=5 ttl=105 time=1.66 ms

--- google.com ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 1.658/1.683/1.718/0.027 ms
ubuntu@ip-10-6-8-13:~$
```

Successfully pinging a website means my VPC is created properly.

1.SSH into my second instance

```
~ $ ssh -i devenv-key.pem ubuntu@44.211.144.92
The authenticity of host '44.211.144.92 (44.211.144.92)' can't be established.
ED25519 key fingerprint is SHA256:GNVxUQHDNP/0mwz1H0t9u/Yo4jokVNRAx8Aaai+F7gw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.211.144.92' (ED25519) to the list of known hosts.
Welcome to Ubuntu 21.04 (GNU/Linux 5.11.0-1007-aws x86_64)
```

```
* Documentation: https://help.ubuntu.com
* Management:   https://landscape.canonical.com
* Support:      https://ubuntu.com/advantage
```

System information as of Sat Mar 29 02:00:50 UTC 2025

```
System load: 0.0          Processes:            101
Usage of /:  17.0% of 7.69GB Users logged in:         0
Memory usage: 20%        IPv4 address for ens5: 10.6.8.5
Swap usage:  0%
```

The list of available updates is more than a week old.
To check for new updates run: `sudo apt update`

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "`sudo <command>`".
See "`man sudo_root`" for details.

```
ubuntu@ip-10-6-8-5:~$
```

```
ubuntu@ip-10-6-8-5:~$ exit
logout
Connection to 44.211.144.92 closed.
~ $
```

2. Create an entry (a rule) in your network ACL

```
~ $ aws ec2 create-network-acl-entry --network-acl-id acl-091a9c3657de4ee08 --ingress --rule-number 80 --protocol tcp --port
-range From=22,To=22 --cidr-block 0.0.0.0/0 --rule-action deny
```

3. Verify if the new rule is added to the default NACL

```

~ $ aws ec2 describe-network-acls --network-acl-ids acl-091a9c3657de4ee08
{
  "NetworkAcls": [
    {
      "Associations": [
        {
          "NetworkAclAssociationId": "aclassoc-06cc48ec7eb62debd",
          "NetworkAclId": "acl-091a9c3657de4ee08",
          "SubnetId": "subnet-09df9a6645aea6ede"
        }
      ],
      "Entries": [
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "allow",
          "RuleNumber": 100
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "deny",
          "RuleNumber": 32767
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": false,
          "PortRange": {
            "From": 22,
            "To": 22
          },
          "Protocol": "6",

```

```

          "CidrBlock": "0.0.0.0/0",
          "Egress": false,
          "PortRange": {
            "From": 22,
            "To": 22
          },
          "Protocol": "6",
          "RuleAction": "deny",
          "RuleNumber": 80
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": false,
          "Protocol": "-1",
          "RuleAction": "allow",
          "RuleNumber": 100
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": false,
          "Protocol": "-1",
          "RuleAction": "deny",
          "RuleNumber": 32767
        }
      ],
      "IsDefault": true,
      "NetworkAclId": "acl-091a9c3657de4ee08",
      "Tags": [],
      "VpcId": "vpc-0cb3e30db617f6175",
      "OwnerId": "881490118823"
    }
  ]
}
~ $ █

```

4.SSH to log into my instance to test the nacl rule created

```
~ $ ssh -i devenv-key.pem ubuntu@44.211.144.92
ssh: connect to host 44.211.144.92 port 22: Connection timed out
~ $
```

5.Remove the rule i added

```
~ $ aws ec2 delete-network-acl-entry --network-acl-id acl-091a9c3657de4ee08 --ingress --rule-number 80
~ $
```

6.Verify if the rule is removed

```
~ $ aws ec2 describe-network-acls --network-acl-ids acl-091a9c3657de4ee08
{
  "NetworkAcls": [
    {
      "Associations": [
        {
          "NetworkAclAssociationId": "aclassoc-06cc48ec7eb62debd",
          "NetworkAclId": "acl-091a9c3657de4ee08",
          "SubnetId": "subnet-09df9a6645aea6ede"
        }
      ],
      "Entries": [
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "allow",
          "RuleNumber": 100
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": true,
          "Protocol": "-1",
          "RuleAction": "deny",
          "RuleNumber": 32767
        },
        {
          "CidrBlock": "0.0.0.0/0",
          "Egress": false,
          "Protocol": "-1",
          "RuleAction": "allow",
          "RuleNumber": 100
        },
        {

```

```

        "RuleAction": "allow",
        "RuleNumber": 100
    },
    {
        "CidrBlock": "0.0.0.0/0",
        "Egress": true,
        "Protocol": "-1",
        "RuleAction": "deny",
        "RuleNumber": 32767
    },
    {
        "CidrBlock": "0.0.0.0/0",
        "Egress": false,
        "Protocol": "-1",
        "RuleAction": "allow",
        "RuleNumber": 100
    },
    {
        "CidrBlock": "0.0.0.0/0",
        "Egress": false,
        "Protocol": "-1",
        "RuleAction": "deny",
        "RuleNumber": 32767
    }
],
    "IsDefault": true,
    "NetworkAclId": "acl-091a9c3657de4ee08",
    "Tags": [],
    "VpcId": "vpc-0cb3e30db617f6175",
    "OwnerId": "881490118823"
}
]
}
~ $

```

7.SSH log into my instance

```

~ $ ssh -i devenv-key.pem ubuntu@44.211.144.92
Welcome to Ubuntu 21.04 (GNU/Linux 5.11.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sat Mar 29 02:15:49 UTC 2025

System load:  0.0               Processes:            101
Usage of /:   17.1% of 7.69GB   Users logged in:     0
Memory usage: 20%              IPv4 address for ens5: 10.6.8.5
Swap usage:   0%

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releaseendoflife

New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Mar 29 02:00:51 2025 from 3.80.78.79
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Last login: Sat Mar 29 02:00:51 2025 from 3.80.78.79
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-6-8-5:~$ exit
logout
Connection to 44.211.144.92 closed.

```

8. Terminate my instances.

```

~ $ aws ec2 terminate-instances --instance-ids i-05746087d8cc178ca
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-05746087d8cc178ca",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
~ $

```

```
~ $ aws ec2 terminate-instances --instance-ids i-07376b78aa6f65f9f
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-07376b78aa6f65f9f",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

Conclusion

I began by setting up a Virtual Private Cloud (VPC) and a subnet inside of it in order to configure my AWS setup. After making sure all the required setups were in place, I started EC2 instances in the subnet. To provide remote access to the instances, I put in place a security group that permitted incoming SSH connections on port 22. A Network Access Control List (NACL) was also configured by me to control traffic at the subnet level. I made sure that the security group and NACL rules were correctly implemented and functioning as planned by trying to connect via SSH after establishing these components.

References

Control traffic to your AWS Resources Using Security Groups - Amazon Virtual Private Cloud. (n.d.-a). <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html>

Control subnet traffic with network access control lists - amazon virtual private cloud. (n.d.-a). <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

Security groups and network ACLS (BP5) - AWS best practices for ddos resiliency. (n.d.-c). <https://docs.aws.amazon.com/whitepapers/latest/aws-best-practices-ddos-resiliency/security-groups-and-network-acls-bp5.html>